

CI/CD Workflow Explanation

Project & Team Task Management Platform

My project utilizes a modern, two-part Continuous Integration and Continuous Deployment (CI/CD) pipeline. This setup ensures that my code is automatically tested for quality (linted and built) before it goes live to my users.

Here is exactly how my workflow operates from the moment I write code to the moment it is deployed.

Part 1: Continuous Integration (GitHub Actions)

My project contains a workflow file located at `.github/workflows/ci.yml`. This file leverages GitHub Actions to act as my primary **Continuous Integration** gatekeeper.

1. Triggers

The workflow is automatically triggered whenever a developer:

- Pushes code directly to the `main` branch.
- Opens or updates a Pull Request targeting the `main` branch.

2. The Execution Environment

GitHub spins up an isolated, temporary server (`ubuntu-latest`) and installs **Node.js v20** to mirror a production environment.

3. Backend Validation Steps

The pipeline changes directories into your `/backend` folder and executes:

- `npm ci`: Installs backend dependencies cleanly from the lockfile.
- `npm test --if-present`: Looks for automated tests and runs them if they exist.
- `npm run build --if-present`: Validates that backend compilation (if any) succeeds.

4. Frontend Validation Steps

The pipeline then moves to your `/frontend` folder and executes:

- `npm ci`: Installs frontend dependencies.
 - `npm run lint`: Runs **ESLint** to enforce code quality, catching syntax errors and messy code styles before they are merged.
 - `npm run build`: Executes `next build`. This is a critical step because Next.js has strict build requirements. If there are TypeScript errors, missing modules, or severe linting violations, **the build will fail and reject my code**.
-

Part 2: Continuous Deployment (Vercel)

While GitHub Actions handles the *Integration* and testing, **Vercel** handles the *Deployment*.

1. Webhook Listening

Vercel is directly connected to my GitHub repository. It constantly listens for any push to the `main` branch.

2. Automatic Building

Once I push code (and assuming my GitHub Actions pass), Vercel automatically clones the repository and begins building it using its highly optimized Next.js and Node.js build systems.

3. Serverless Distribution

- **Frontend:** Vercel deploys my Next.js application globally across its Edge Network, ensuring lightning-fast load times.
- **Backend:** Because of the `vercel.json` configuration file I have in my backend, Vercel intelligently takes my `server.js` Express app and wraps it into **Serverless Functions**.

4. Zero-Downtime Releases

When the build is successfully completed on Vercel, it atomically swaps the old version of my site with the new version. If Vercel encounters a build error (such as a typo breaking the app), it **cancels the deployment** and keeps the old working version live, ensuring my users never see a broken application.

Summary of the Flow

Write Code → Push to GitHub → GitHub Actions runs Lint & Build (CI) → Vercel auto-deploys to Serverless Functions (CD)